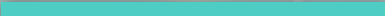


Kimi Team Technical Report

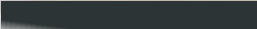
Attention Residuals



A Novel Approach to Depth-Wise Information Aggregation in
Transformers



github.com/MoonshotAI/Attention-Residuals



CONTENTS

01

Introduction & Problem

Standard residual limitations and motivation

02

Attention Residuals

Full and Block architecture design

03

Infrastructure Design

Training and inference optimizations

04

Experimental Results

Scaling laws and benchmarks

05

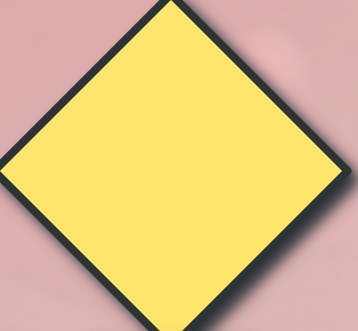
Theoretical Insights

Sequence-depth duality and matrices

06

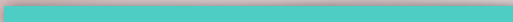
Related Work

Comparison with residual variants

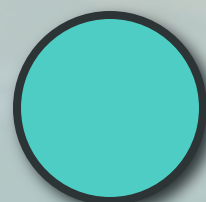
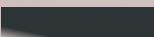


01

Introduction & Problem Statement

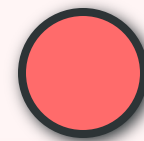


Understanding the limitations of standard residual connections
and the motivation for Attention Residuals





The Problem with Standard Residuals



1

Standard Residual Update

$$h_i = h_{i-1} + f_{i-1}(h_{i-1})$$

PreNorm paradigm accumulates all layer outputs with **fixed unit weights**

2

Unrolling the Recurrence

$$h_i = h_1 + \sum_{j=1}^{i-1} f_j(h_j)$$

Every layer receives **uniformly-weighted** sum of all prior outputs



Core Issue

Unlike sequence mixing and expert routing with **learnable input-dependent weighting**, depth-wise aggregation uses **fixed weights**—no mechanism to selectively emphasize layers.



PreNorm Dilution Problem



Hidden-state magnitudes grow linearly with depth



Each layer's **relative contribution shrinks** progressively



Early-layer information is buried and cannot be retrieved



Empirical Evidence

A significant fraction of layers can be **pruned with minimal loss**—indicating that current residual connections fail to effectively utilize depth.

■ Limited depth utilization



The situation parallels RNN challenges before attention mechanisms





Key Limitations of Current Approaches



1 No Selective Access

Each layer only accesses h_{i-1} —a single compressed state conflating all earlier outputs.

Problem: Attention & MLP layers receive the **same aggregated state** despite needing different weightings.

2 Irreversible Loss

Information lost through aggregation cannot be selectively recovered in deeper layers.

Problem: Early layer outputs are **permanently buried** in the cumulative sum.

3 Output Growth

Later layers learn larger outputs to gain influence over the accumulated residual.

Problem: This can **destabilize training** and limits effective depth utilization.



Generalizing Residuals

$$h_i = \alpha_i \cdot h_{i-1} + \beta_i \cdot f_{i-1}(h_{i-1})$$

Weighted recurrence generalizes both approaches

Residual:

$$\alpha_i = \beta_i = 1$$

Highway:

$$\alpha_i = 1 - g_i, \beta_i = g_i$$

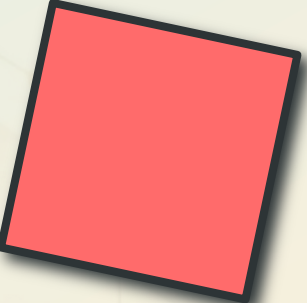


Fundamental Constraint

Both approaches share a critical limitation:

Each layer can only access its **immediate input** h_{i-1} —a single compressed state that conflates all earlier layer outputs. **Not the individual outputs themselves.**

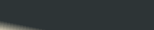
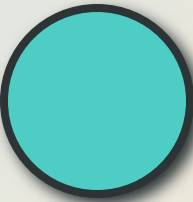
This motivates a mechanism for selective aggregation from all preceding layers.



02

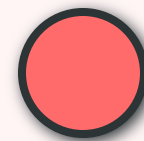
Attention Residuals Architecture

Introducing the core methodology: replacing fixed accumulation with learned depth-wise attention





The Time-Depth Duality



💡 The Core Insight

Formal duality: Residual connections compress all prior information into a single state **over depth**, just as RNNs compress information **over time**.



Sequence (Time)

RNN: $h_t = f(h_{t-1}, x_t)$

Recurrent update over time steps

Transformer Solution:

Replace recurrence with **self-attention** —each position selectively accesses all previous positions.

✓ Revolutionized sequence modeling



Depth (Layers)

Res: $h_i = h_{i-1} + f_{i-1}(h_{i-1})$

Residual update over layers

AttnRes Solution:

Apply the **same methodology** —each layer selectively aggregates all previous layer outputs.

→ Completes the linear-to-softmax transition for depth

$O(L^2)$

Feasible —depth typically $L < 1000$



Learned input-dependent weights



Selective information retrieval



Full Attention Residuals



Core Equation

$$h_i = \sum_{j=0}^{i-1} \alpha_{i \rightarrow j} \cdot v_j$$

Attention-weighted sum over all previous outputs

where $\alpha_{i \rightarrow j}$ are layer-specific attention weights with $\sum \alpha_{i \rightarrow j} = 1$

2 Attention Weights

$$\alpha_{i \rightarrow j} = \phi(q_i, k_j) / \sum_j \phi(q_i, k_j)$$

Softmax attention over depth

Kernel Function:

$$\phi(q, k) = \exp(q^T \text{RMSNorm}(k))$$

RMSNorm prevents layers with large outputs from dominating.

1 Query & Values

Query

$$q_i = w_i$$

Layer-specific learnable vector in $\mathbb{R}^d$

Key/Value

$$k_i = v_i = f_i(h_i)$$

Layer outputs ($v_0 = h_1$ embedding)

3 Complexity Analysis

$O(L^2d)$

Arithmetic

$O(Ld)$

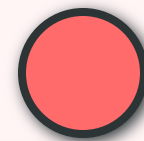
Memory

Modest cost —depth is far smaller than sequence length

i Memory overlaps with backprop activations—no extra overhead in vanilla training.



Block Attention Residuals



🔑 Scalable Variant for Large Models

Full AttnRes requires $O(Ld)$ communication across pipeline stages. Block AttnRes reduces this to $O(Nd)$ by partitioning layers into N blocks.

1 Block Structure

L layers $\rightarrow$ N blocks of $S = L/N$ layers each



Intra-Block Accumulation:

$$b_n = \sum_{j \in B_n} f_j(h_j)$$

Sum all layer outputs within block

Inter-Block Attention:

Attend over N block representations + embedding

2 Efficiency Gains

Memory

$$O(L) \rightarrow O(N)$$

Compute

$$O(L^2) \rightarrow O(N^2)$$

Block Count N :

- $N = L$: Full AttnRes
- $N = 1$: Standard residual
- $N \approx 8$: Recovers most gains

✓ Empirically: $N \approx 8$ requires only 8 stored hidden states per token

</> Value Matrix Configuration

First Layer of Block:

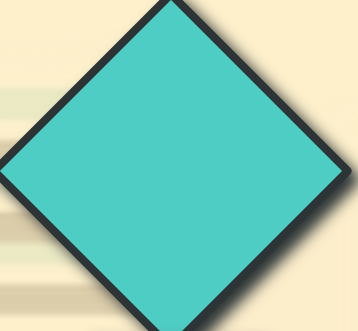
$$V = [b_0, b_1, \dots, b_{n-1}]^T$$

Previous blocks + embedding

Subsequent Layers:

$$V = [b_0, \dots, b_{n-1}, b_n^{i-1}]^T$$

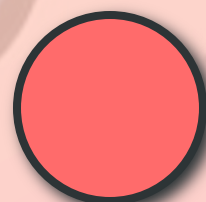
+ partial sum within block



03

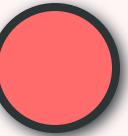
Infrastructure Design

System optimizations that make Block AttnRes practical and
efficient at scale





Training: Cross-Stage Caching



⚠ Pipeline Communication Challenge

Block AttnRes requires all accumulated block representations at each stage for inter-block attention. Naïve transmission incurs **heavy redundant communication**.

✗ Naïve Approach

Transmit full history at every transition:

$$\text{Comm}_{\text{naïve}} = \sum_{j=1}^{C-1} j N_p d$$

Result:

$O(C^2)$ communication cost — **prohibitive at scale**

📈 Peak cost: $O(C)$ per transition

✓ Cross-Stage Caching

Key Insight:

Each physical stage processes multiple virtual stages — **cache blocks locally** to avoid re-transmission

Transmit only incremental blocks:

$$\text{Comm}_{\text{cached}} \approx (V-1)P^2 N_p d$$

↓ Peak cost reduced to $O(P)$ — $V \times$ improvement!

🏗 Pipeline Example: $P=4, V=2$

Rank 0

$[b_0]$

$[+b_1, b_2]$

Rank 1

$[b_0, b_1]$

$[+b_2, b_3]$

Rank 2

$[b_0, b_1, b_2]$

$[+b_3, b_4]$

Rank 3

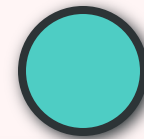
$[b_0, b_1, b_2, b_3]$

$[+b_4]$

★ Caching eliminates 6 redundant block transmissions for 2nd virtual stage



Inference: Two-Phase Strategy



Key Insight

Pseudo-query w_i is decoupled from forward computation—all S queries within a block can be **batched** into single matrix multiplication, amortizing memory access.

1

Phase 1: Inter-Block

Parallel Computation:

Compute attention for **all S layers simultaneously** against cached block representations

Returns:

Outputs + softmax statistics (max, log-sum-exp)



Amortizes reads from S times to **once** per block

2

Phase 2: Intra-Block

Sequential Processing:

Walk through layers **in order** using evolving partial sum

Online Softmax Merge:

Merge Phase 1 outputs with intra-block attention via **online softmax**



Admits **kernel fusion** with surrounding operations



Memory Access Cost Per Layer

Standard

3d

Total I/O

mHC (m=4)

34d

Total I/O

Full AttnRes

24d

Total I/O

Block AttnRes

5.5d

Total I/O



Training overhead: <4%



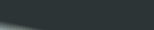
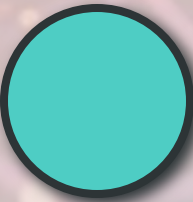
Inference overhead: <2%



04

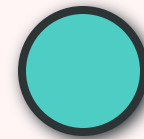
Experimental Results

Comprehensive validation through scaling laws, training dynamics, and downstream benchmarks

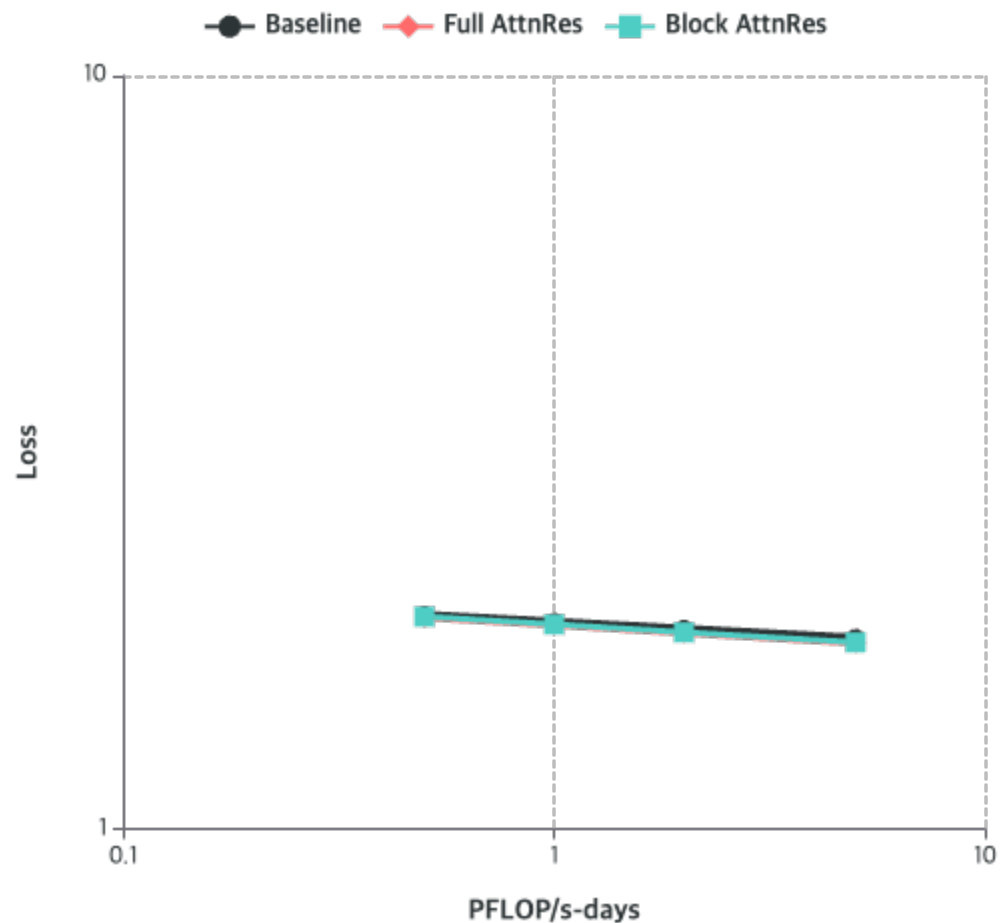




Scaling Laws: Consistent Improvements



Scaling Law Curves



🔄 Fitted Power Laws

Baseline: $L = 1.891 \times C^{-0.057}$

Full AttnRes: $L = 1.865 \times C^{-0.057}$

Block AttnRes: $L = 1.870 \times C^{-0.058}$

★ Key Findings

Consistent Gains:

AttnRes achieves **lower loss across all compute ranges**

1.25× Compute Advantage:

Block AttnRes at 5.6 PFLOP/s-days matches baseline at 7 PFLOP/s-days

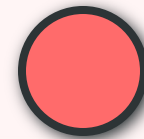
Gap Narrows with Scale:

Full vs Block difference shrinks to just **0.001** at largest size

i 5 model sizes: 194M to 528M active params, 38.7B to 119B tokens



Training Dynamics Analysis



Validation Loss

AttnRes Advantage:
Consistently lower loss throughout training

↘ Gap widens during decay phase



Output Magnitude

Baseline Problem:
 $O(L)$ growth with depth (PreNorm dilution)

✓ AttnRes: bounded periodic pattern



Gradient Magnitude

Baseline Issue:
Disproportionately large gradients in early layers

✓ AttnRes: uniform distribution



Key Observations



Baseline Issues

1. Hidden-state magnitudes grow **monotonically** with depth
2. Deeper layers forced to learn **increasingly large outputs**
3. **No mechanism** to regulate gradient flow

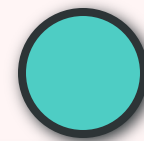


AttnRes Solutions

1. Growth **confined within each block** via selective aggregation
2. Block boundaries **reset accumulation** periodically
3. Learned softmax weights introduce **competition** among sources



Downstream Performance: Kimi Linear 48B

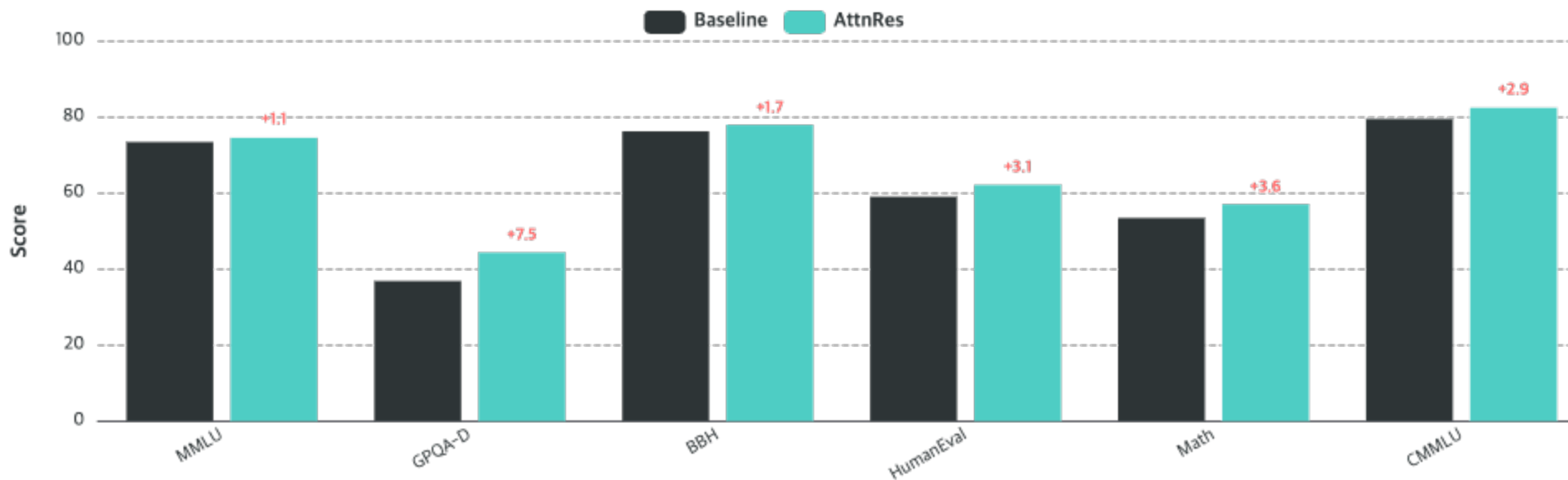


Model Configuration

48B total / 3B activated params | 27 blocks (54 layers) | 9 blocks + embedding

1.4T tokens

Benchmark Results



Reasoning

GPQA-D: +7.5 | Math: +3.6



Code

HumanEval: +3.1 | MBPP: +0.9

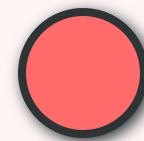


Knowledge

MMLU: +1.1 | TriviaQA: +1.9



Ablation Studies: Design Choices



Comparison with Prior Methods (16-layer model)

Baseline 1.766 PreNorm	DenseFormer 1.767 No gain	mHC 1.747 -0.019	Block AttnRes 1.746 -0.020	Full AttnRes 1.737 -0.029
-------------------------------------	--	-------------------------------	---	--



Component Ablations

Input-dependent query	1.731
Input-independent mixing	1.749
Sigmoid (vs softmax)	1.741
Multihead (H=16)	1.752
w/o RMSNorm (Full)	1.743
w/o RMSNorm (Block)	1.750



Key Insights

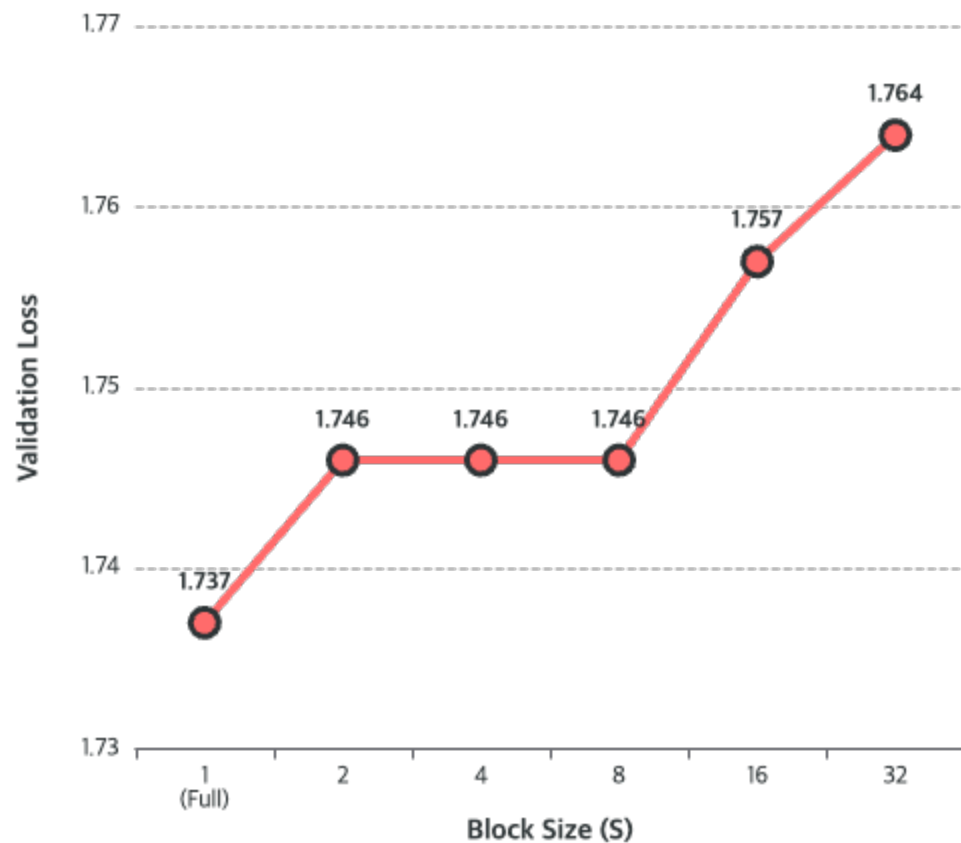
Input-dependence matters: DenseFormer (static) shows no gain vs baseline
Softmax > Sigmoid: Competitive normalization forces sharper selection
Single query suffices: Multihead hurts—optimal depth mixture is uniform across channels
RMSNorm is critical: Prevents magnitude differences from biasing attention



Block Size Analysis



Effect of Block Size (16-layer model)



↔ Block Size Spectrum

S=1 Full AttnRes (1.737)

S=2,4,8 Optimal range (~1.746)

S=16,32 Toward baseline (1.757-1.764)

🎯 Key Observations

Graceful degradation: Loss increases gradually as S grows

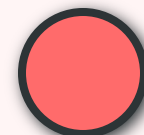
Sweet spot: S=2,4,8 all achieve ~1.746 with minimal variation

Practical choice: N≈8 blocks for infrastructure efficiency

🔗 Finer-grained blocking is a promising pathway as hardware improves



Optimal Architecture: Depth-Width Trade-off



Controlled Capacity Reallocation Study

Fixed Budget:

FLOPs $\approx 6.5 \times 10^9$, Params $\approx 2.3 \times 10^8$

Grid Search:

5x5 over $d_{mo}d_{el}/L_\beta$ and H/L_β

Goal:

Does AttnRes alter depth-width preference?

Baseline Optimum

1.973	1.883	1.859	1.849	1.854
1.917	1.841	1.819	1.812	1.817
1.952	1.868	1.850	1.849	1.857
1.893	1.823	1.815	1.813	1.813
1.926	1.857	1.851	1.847	1.858

Optimum: $d_{mo}d_{el}/L_\beta \approx 60$ (1.847)

AttnRes Optimum

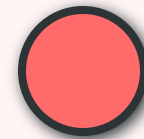
1.931	1.863	1.830	1.817	1.818
1.883	1.841	1.819	1.812	1.817
1.859	1.850	1.849	1.847	1.857
1.849	1.847	1.847	1.846	1.858
1.854	1.858	1.858	1.858	1.858

Optimum: $d_{mo}d_{el}/L_\beta \approx 45$ (1.802)

★ Key Finding: AttnRes shifts optimum to lower $d_{mo}d_{el}/L_\beta$ (deeper, narrower), suggesting it can exploit additional depth more effectively



Learned Attention Patterns



Visualization of learned weights $\alpha_i \rightarrow_i$ for 16-layer model. Each row shows how layer l distributes attention over previous sources.

1

Preserved Locality

Each layer attends most strongly to its immediate predecessor

Selective off-diagonal concentrations emerge (learned skip connections)

Example: layer 4 attends to early sources

2

Layer Specialization

Embedding h_1 retains non-trivial weight throughout

Pre-attention layers: broader receptive fields

Pre-MLP layers: sharper diagonal reliance

3

Block Preserves Structure

Diagonal dominance transfers from Full to Block

Embedding persistence and layer specialization preserved

Block acts as implicit regularization



What the Model Learns

Local + Global

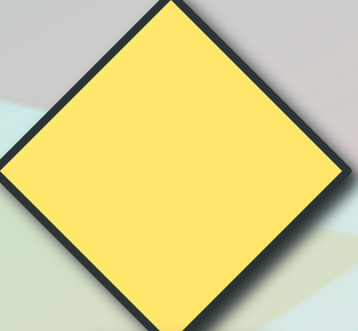
Combines immediate predecessor with selective long-range access

Task-Specific

Different layer types develop different attention patterns

Compressed but Effective

Block variant maintains essential pathways

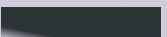


05

Theoretical Insights

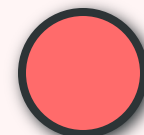


Understanding Attention Residuals through sequence-depth
duality and structured matrices





Sequence-Depth Duality



Formalizing the Duality

Sequence (TTT):

$$W_t = W_{t-1} - \eta \nabla \ell(W_{t-1}; x_t)$$

Gradient descent on self-supervised loss

Depth (Residual):

$$h_l = h_{l-1} + f_{l-1}(h_{l-1})$$

Each layer as one "gradient step"



Method Correspondence

Sequence Method	Depth Method	Mechanism
Data-dependent gates	Highway Networks	Gating mechanism
Delta rule	DDL	Erase-and-write update
GLA (Gated Linear Attn)	MRLA	Gated linear attention
Softmax Attention	AttnRes (ours)	Content-dependent selection

Prior Methods:

All refine the **recurrent update** within recurrence paradigm

AttnRes Innovation:

Replaces recurrence with **direct cross-layer attention**

Residual Connections as Structured Matrices

↻ Depth Mixing Matrix $M \in \mathbb{R}^{(L \times L)}$

$M_{i \rightarrow j}$ = weight that layer j assigns to layer i . Input to layer j : $h_j = \sum_{i=0}^{j-1} M_{i \rightarrow j} v_i$

Methods as Matrix Structures

Method	Matrix Structure	Rank
Standard Residual	All-ones lower-triangular	1
Highway	1-semiseparable (gated)	1
(m)HC	m-semiseparable	m
Full AttnRes	Dense, input-dependent	L
Block AttnRes	Block-structured	N to N+S

Analytical Insights:

Reveals depth-wise attention sinks (like sequence attention sinks)

Design Guidance:

Exposes which kernel properties matter for new designs

Prior Residuals as Depth-Wise Linear Attention

The (m)HC-Attention Connection

$$M_{i \rightarrow i} = \beta_i^T A \times_{(i+1)} \rightarrow_i \alpha_i$$

Natural attention interpretation: α_i as query, β_i as key, cumulative transition as depth-relative positional operator

Linear Attention

Kernel decomposition:

$$\phi(q, k) = \varphi(q)^T \varphi(k)$$

(m)HC acts as depth-wise linear attention with matrix-valued states

m parallel streams = **state expansion** along depth

Softmax Attention

AttnRes kernel:

$$\phi(q, k) = \exp(q^T \text{RMSNorm}(k))$$

AttnRes acts as depth-wise softmax attention

★ Completes the linear-to-softmax transition for depth

Completing the Picture

Sequences

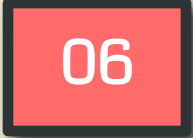
RNN $\rightarrow$ Linear Attention $\rightarrow$ Softmax Attention

Depth

Residual $\rightarrow$ (m)HC $\rightarrow$ AttnRes

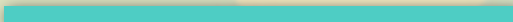
Unified View

Same progression, different dimensions

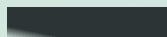
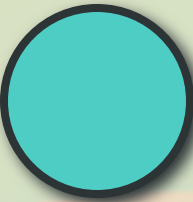


06

Related Work

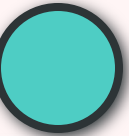


Positioning Attention Residuals within the landscape of residual
connection research





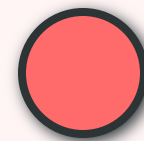
Comparison with Residual Variants



Category	Method	Weight Type	Source Access
Single-State Recurrence	Residual	Fixed	h_{i-1} only
	ReZero	Static	h_{i-1} only
	LayerScale	Static	h_{i-1} only
	Highway	Dynamic	h_{i-1} only
	DeepNorm	Fixed	h_{i-1} only
	KEEL	Fixed	h_{i-1} only
Multi-State Recurrence	SiameseNorm	Fixed	2 streams
	HC/mHC	Dynamic	m streams
	DDL	Dynamic	d_v streams
Cross-Layer Access	DenseNet	Fixed	$[h_1, \dots, h_{i-1}]$
	DenseFormer	Static	$[h_1, \dots, h_{i-1}]$
	MRLA	Dynamic	$[h_1, \dots, h_{i-1}]$
	AttnRes (ours)	Dynamic	$[h_1, \dots, h_{i-1}]$ or blocks



Key Differentiators



1

vs Normalization/Scaling

PreNorm/PostNorm Tension:
Prior methods try to **reconcile** the trade-off

AttnRes Solution:
Sidesteps the tension via selective aggregation

Avoids **cumulative magnitude growth** and **repeated scale contraction**

2

vs Multi-State Recurrence

(m)HC, DDL, SiameseNorm:
Maintain **multiple parallel streams**

Still Limited:
Condition on **immediate predecessor state** only

AttnRes Advantage:
Selective access to individual earlier-layer outputs

3

vs Cross-Layer Connectivity

DenseFormer, MRLA:
Provide **cross-layer access** but with limitations

Their Limitations:
Static weights or **linear attention** (not softmax)

AttnRes Uniqueness:
Softmax-normalized + input-dependent + scalable



AttnRes: The Complete Package

Content-Dependent

Learned softmax weights adapt to input

Selective Access

Retrieve from any previous layer

Scalable

Block structure reduces $O(L)$ to $O(N)$

Practical

<4% training, <2% inference overhead

CONCLUSION

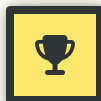
Attention Residuals: A New Paradigm for Depth-Wise Aggregation

Attention Residuals replaces fixed, uniform residual accumulation with **learned, input-dependent depth-wise attention**



Scaling Laws

Consistent gains across all scales



Benchmarks

Improves on all evaluated tasks



Practical

<4% training, <2% inference

 github.com/MoonshotAI/Attention-Residuals